



WEEK 1 • COURSE KICKOFF

AI Augmented Software Engineering

Scott Davis, Muratcan Cicek, Sedanur Ceylan, Dr. Muhammed Davud

WELCOME

Your teaching team

SD

INSTRUCTOR

Scott Davis

University of California, Santa Cruz

SC

INSTRUCTOR

Sedanur Ceylan

Founder, Dream Data Labs

MD

INSTRUCTOR

Dr. Muhammed Davud

Istinye University

MC

INSTRUCTOR

Dr. Muratcan Cicek

University of California, Santa Cruz

Questions outside class: email any of us.

WHERE I AM COMING FROM

Santa Cruz, California

On the Pacific coast

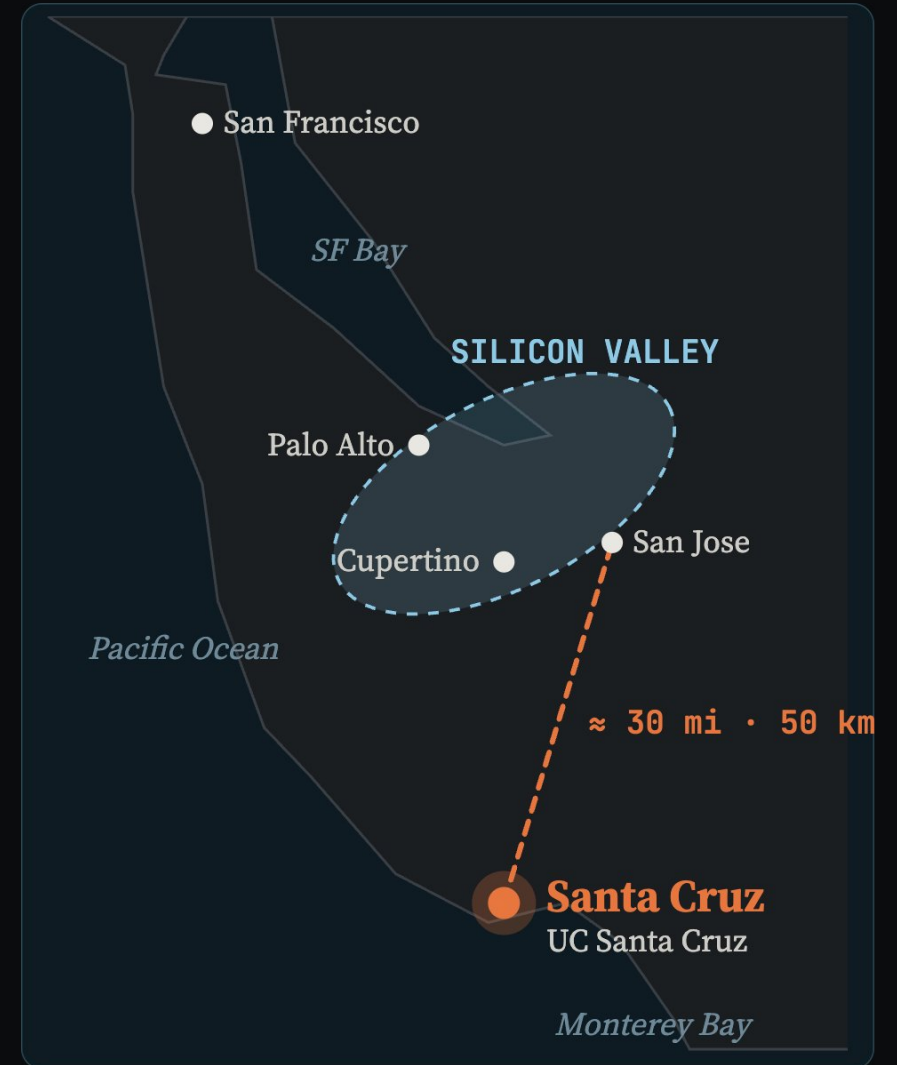
A beach town on Monterey Bay, south of San Francisco.

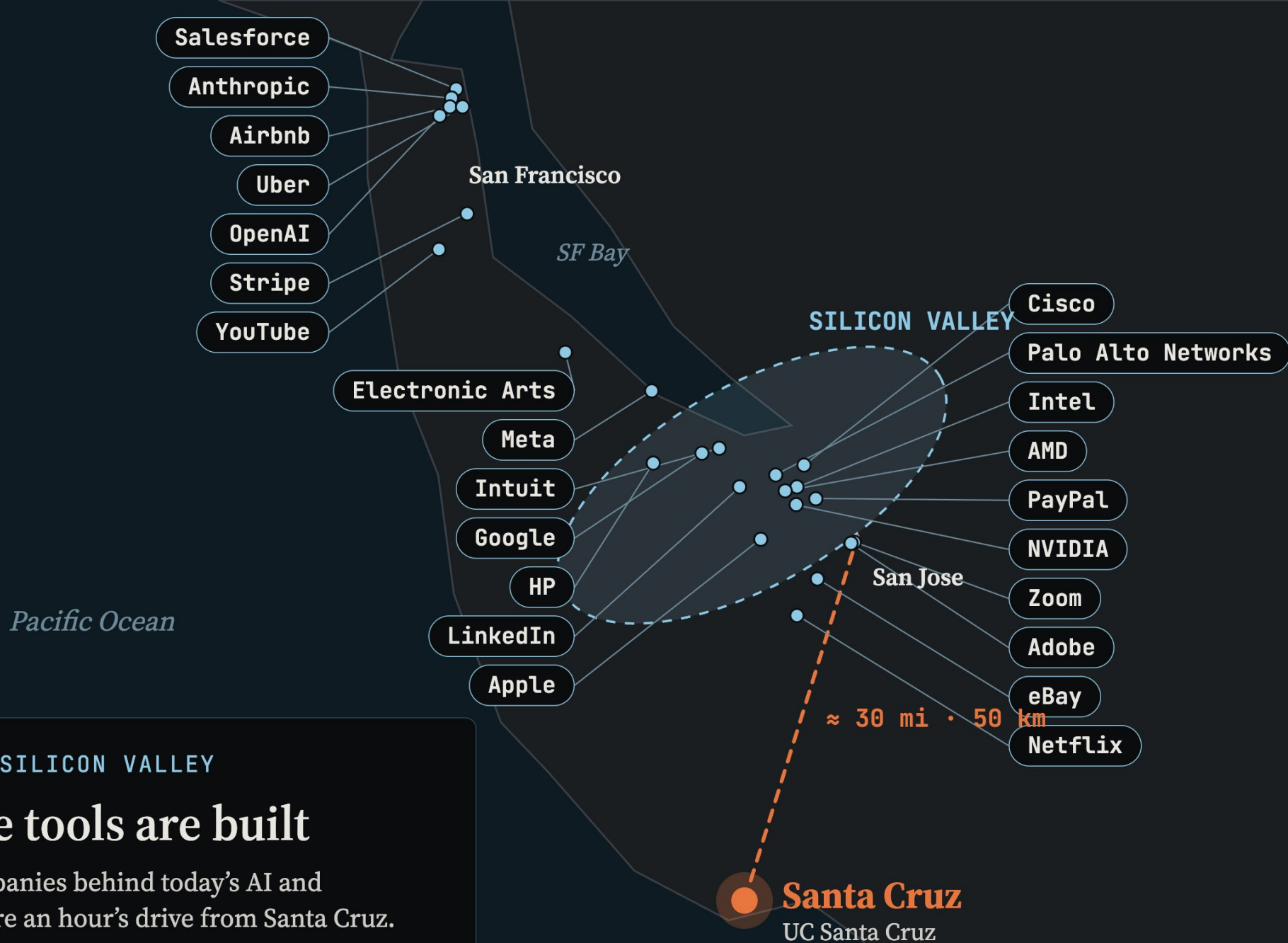
Next door to Silicon Valley

About 30 miles (≈ 50 km) from San Jose, over the Santa Cruz Mountains.

Home of UC Santa Cruz

Where two of your instructors from.





NEXT DOOR TO SILICON VALLEY

Where the tools are built

Many of the companies behind today's AI and developer tools are an hour's drive from Santa Cruz.

ABOUT ME

Scott Davis

INDUSTRY **15 years in software engineering**

Building and shipping software products.

STARTUPS **Advisor to several startups**

Including Arena (arena.ai).

TEACHING **Lecturer at UC Santa Cruz**

Software Engineering in the Age of AI · the CSE 115 series, a year-long capstone project.

RESEARCH **Researcher, Hecate Lab at UCSC**

HCI · AI and education · LLM benchmarking.



Arena · one of the startups I advise

YOUR INSTRUCTORS

Dr. Muratcan Cicek

EDUCATION PhD, UC Santa Cruz

RESEARCH Google Scholar

INDUSTRY Amazon · eBay

FOUNDER CEO of Yeterly

YOUR INSTRUCTORS

Sedanur Ceylan

FOUNDER

CEO of Dream Data Labs

Leads patent-backed AI memory models for learning and knowledge.

RESEARCH

AI researcher & software engineer

Memory models · agentic systems · AI-augmented software engineering.

INDUSTRY

100+ software projects

Istanbul Finance Center · Atatürk Cultural Center · Hayat Kimya.

EDUCATION

Computer Engineering, Sakarya University

Teaches AI and software engineering to companies.

What this course covers

The emerging practices and technologies behind AI-native software development:

MCP

Agent skills

Spec-driven development

Loop engineering

The software factory

YOU WILL LEARN HOW TO

- Provide agents with the right context and capabilities
- Translate product requirements into executable specifications
- Design iterative workflows in which humans and agents plan, build, evaluate, and improve software together

How the course works

- Hands-on assignments
- Team projects
- Discussion, reading cutting edge research papers

BY THE END, YOU WILL BE ABLE TO

- Design effective agent-driven workflows
- Compose tools and skills into reliable development systems
- Apply software-factory principles to build and evolve software at greater speed and scale

How you'll learn

01 Lectures

Weekly sessions with live demos

02 Weekly assignments

Hands-on practice with coding agents

03 Team projects

Build a real product as a team of 5

04 Reading material

Articles and guides for each topic

05 Research papers

The work behind today's agents

06 Videos

Talks and walkthroughs to watch

Most importantly, learn
from each other.

How every week runs

01 Readings

Videos, papers and articles to go through before class.

02 Lecture & discussion

The week's topic in-class discussion.

03 Assignment

Put the topic into practice, hands-on in your coding agent.

04 Project

Apply it to your team's project and move your sprint forward.

Where to find everything

Course site

Syllabus, schedule, policies and FAQ.

`ai-augmented-se-production.up.railway.app`

GitHub repo

Lecture notes, weekly assignments and starter code.

`github.com/scottyUX/
AI-Augmented-Software-Engineering`

Grepthink2

Post project ideas, join a team, track your project.

`grepthink2.com` – opens Monday

Weekly task submissions

Each week, group members submits a task from your team project.

`Site address coming soon`

Attendance %5

1 absence is automatically excused. No need to tell us

After that, unexcused absences will penalize your participation score

Attendance is required for **all** students at their team presentations

Participation %5

Participate

Join discussions, labs and the end-of-class Q&A.

Ask questions

If you can't hear or follow something, tell us right away.

Use agents freely. You own every line you submit.

- Assignments and the project: coding agents are expected.
- You must be able to explain any code you submit.
- The midterm is your own work.
- No AI for reflections

MIDTERM • 30%

The midterm

1 hr

In class, in midterm week

30%

Of your final grade

Wk 6

Most likely — date to
be confirmed

WHAT IT COVERS

Everything we cover in the lectures and in the required reading materials.

GRADING

How your grade is built

5%

Attendance

5%

Participation

20%

Assignments

30%

Midterm

40%

Final project

Lecture Schedule

WEEK 1	The AI-Native Software Engineer
WEEK 2	Spec-Driven Development
WEEK 3	Anatomy of Coding Agents
WEEK 4	Advanced Context Management
WEEK 5	Agentic Codebase Structure
WEEK 6	Agentic Code Review and Testing MIDTERM • 1 HOUR

WEEK 7	Agent Orchestration
WEEK 8	Security in the Age of AI
WEEK 9	AI-Native Engineering Teams
WEEK 10	Final Presentations
WEEK 11	Final Presentations

The AI-Native Software Engineer

01 The Vibe-Coding Trap

02 What AI-Native Engineering Is Not

03 The Role Transformation: From Implementer to Orchestrator

04 Human-in-the-Loop: The Nonnegotiable Checkpoint

05 What AI-Native Engineers Actually Do

06 Career Implications

07 Junior Software Engineers in the Age of AI

Spec-Driven Development

01 The Agentic Software Development Life Cycle

02 Why Do We Need SDD?

03 When SDD Is Not the Right Fit

04 SDD and Human-in-the-Loop

05 Plan Mode: The Bridge Between Vibe Coding
and SDD

06 Three Levels of SDD Maturity

07 What Good Specs Look Like

08 The SDD Workflow

09 SDD Frameworks

Anatomy of Coding Agents

- 01 How LLMs work
- 02 The model landscape
- 03 Why LLMs are good at coding
- 04 Coding agents under the hood

Advanced Context Management

01 A Taxonomy of Context Components

02 The Context Assembly Process

03 Context Discovery

04 The Agent Runtime Pipeline

05 Session Management

06 MCP fundamentals: servers, clients, tools,
and transport

Agentic Codebase Structure

- 01 Skills, rules, and guardrails
- 02 CLAUDE.md, AGENTS.md, SKILL.md, intent.md
- 03 Hooks for lint gates and test runs
- 04 Subagent patterns (planner / implementer / reviewer)

Agentic Code Review and Testing

01 Deterministic Guardrails

02 LLM-based review

03 Safe Deployment Strategies

04 Runtime Safety and AI-Powered Ops

05 The Human in the Loop

06 Code review tools and frameworks

07 Monitoring Agents

Agent Orchestration

01 Generator-Critic Loop

02 The Goal Pattern

03 The Ralph Loop

04 Running multiple agents

05 Subagent Pattern

Security in the Age of AI

- 01 SAST / SCA, dependency and secret-leak vulnerabilities
- 02 Prompt injection and agent-specific attack surfaces
- 03 Agent-assisted triage and remediation

AI-Native Engineering Teams

- 01 How AI-native teams work
- 02 A standardized team stack
- 03 LLM gateways, model routing, and cost optimization

One hands-on assignment per lecture

Each assignment turns that week's topic into practice — prompting, specs, an MCP server, code review, orchestration, security.

Briefs and starter code are in the GitHub repo.
Submit a link to your repo.

THIS WEEK · ASSIGNMENT 1

Set up your AI-native environment

- Install Git, Python 3.12 and an editor
- Get a coding agent working (Antigravity + Gemini, or your own)
- Have the agent build a small program; push it with a writeup

Pass/fail · 20 pts · about 2–3 hours · due [date]

Form your teams

START HERE Grepthink2 Post a project idea or join one. Teams of 5 by Wednesday. grepthink2.com	KICKOFF Initial presentation 8 min + 2 min Q&A. Mandatory, not graded.	EVERY 2 WEEKS Sprint demo 5 minutes on what your team shipped. Mandatory, not graded.	WEEKS 10-11 Final presentation Graded. Every team presents its finished project.
--	--	---	--

The final presentation shows:

a working demo · your architecture decisions · the skills and rules you designed · your security measures

How teams form on Grepthink

01

Propose an idea

Whoever proposes a project usually becomes its product owner. It doesn't need to be fully figured out.

02

Explore and ask to join

Look at several projects and talk to their product owners before you decide.

03

Commit to one

Once you join a team, let the other projects know you're taken.

04

Reach exactly 5

Every team has five members by Wednesday.

TEAMS

The next eight days

MONDAY	September 28	Grepthink2 opens — register, get the join code
WEDNESDAY	September 30	Teams of 5 formed
FRIDAY	October 2	Initial presentations: 8 min + 2 min Q&A
THEN	Every two weeks	Sprint ends with a 5-minute team demo

Choose teammates you can actually meet with

Schedules first

Clashing schedules sink more teams than anything else. Check overlap before you commit and set a weekly meeting.

Friends or new people

Friends are an easy start but harder to hold to account. New teammates take longer to warm up but often stay focused.

Agree up front

How you'll communicate, when work is due, and what happens when someone falls behind.

Software Engineering in the *Age of AI*.

*Why software engineering fundamentals matter
more than ever*

AI

“It’ll be 10 times bigger than the Industrial Revolution — and maybe 10 times faster.”

Demis Hassabis · CEO of DeepMind

AI

The most profound shift in software engineering isn't a new language, framework, or cloud service. It's the transition from **writing code to expressing intent**, and trusting intelligent systems to translate that intent into working software.

The New SDLC With Vibe Coding · Google

WATCH • STANFORD ONLINE



Andrew Ng: One Skill to Stay Relevant in the Age of AI

youtube.com/watch?v=HTlf3KNNez0

LIVE POLL • LECTURE 1

How are you feeling about AI as a Junior Engineer?

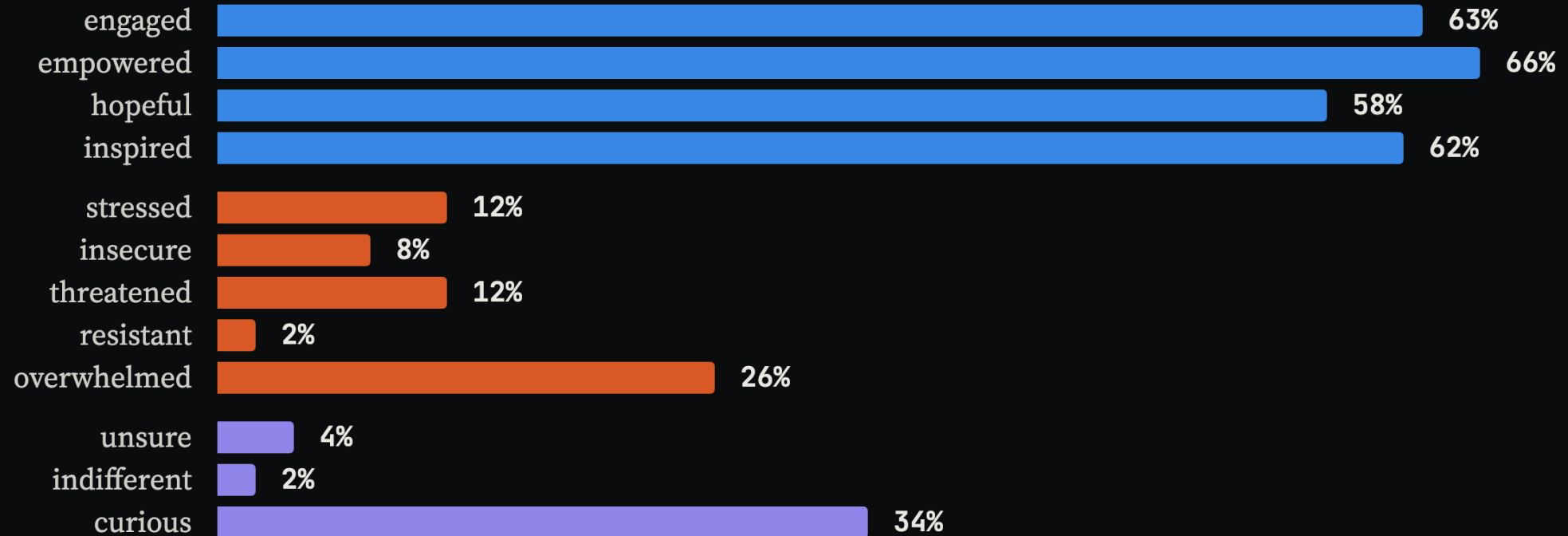
Go to **menti.com** and enter the code

3608 4172



Percent of Googlers who respond to each emotion

■ Positive ■ Negative ■ Neutral



Setting the scene

90%

of software engineers use AI
at work

40%

rely on it a lot

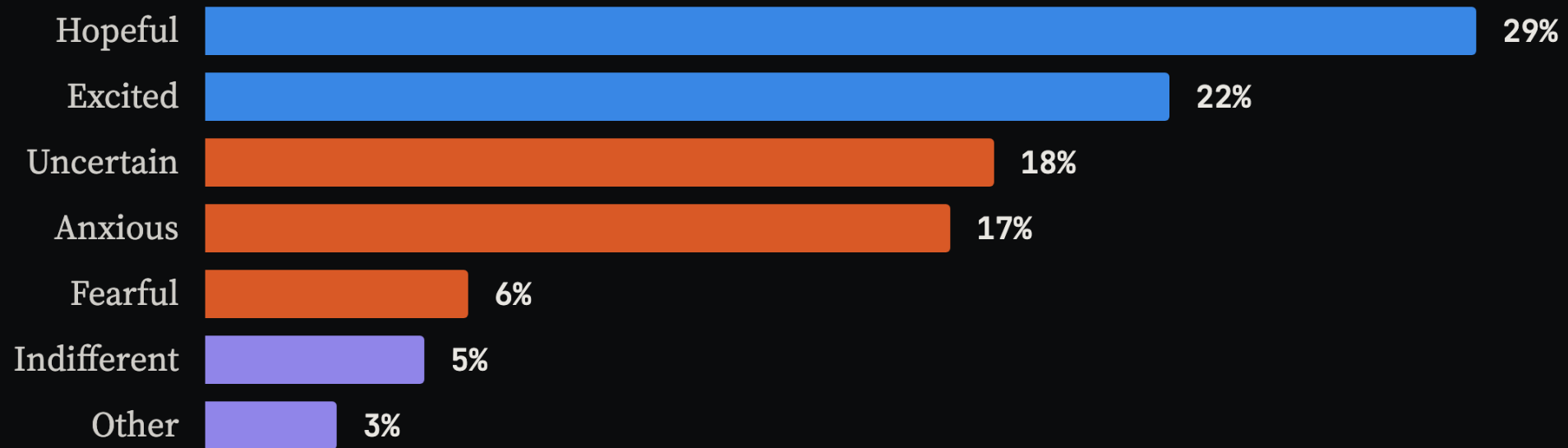
3/4

of code at Google is
AI-generated

Source: DORA State of AI-assisted Software Development (2025) & Google Research

How developers feel about the increasing role of AI in society

■ Positive ■ Negative ■ Neutral



Source: JetBrains Developer Ecosystem Survey 2025 – 24.5K developers, 194 countries

How AI impacts skill formation

Warm up 10 min	Trio task 35 min	Post quiz 25 min	Post survey 5 min
TREATMENT No AI	TREATMENT ⚡ AI assist	TREATMENT No AI	TREATMENT No AI
CONTROL No AI	CONTROL No AI	CONTROL No AI	CONTROL No AI

We recruited 52 (mostly junior) software engineers, each of whom had been using Python at least once a week for over a year. We also made sure they were at least somewhat familiar with AI coding assistance, and were unfamiliar with Trio, the Python library on which our tasks were based.

Source: Anthropic, "How AI assistance impacts the formation of coding skills," 2026

The results: speed vs. skill

Coding speed

~2 min faster

Not statistically significant

Knowledge score

AI-ASSISTED

50%

NO AI

67%

17 percentage points lower

Statistically significant ($p = 0.010$)

Bottom line: AI may help you finish a task faster, but may reduce how much you learn from doing it.

Source: Anthropic, "How AI assistance impacts the formation of coding skills," 2026

Software engineering vs vibe coding vs programming

The Vibe Coding to Agentic Engineering Spectrum

WATCH • IBM TECHNOLOGY • 10 MIN



Agentic Engineering vs Software Engineering: Beyond Vibe Coding

youtube.com/watch?v=FgaBdwSv0GM

THE SPECTRUM

The differentiator is not whether you use AI — it's **how outputs get verified.**

Vibe coding with AI

Casual prompts

“Does it seem to work?”

Disposable code

Structured AI-assisted

Detailed prompts + constraints

Manual testing + spot checks

Features in established codebases

Software engineering

Formal specs + architecture docs

Automated evals + CI/CD gates

Production systems at scale

← Less structure, more speed

More structure, more reliability →

The spectrum in detail.

Dimension	Vibe Coding	Structured AI-Assisted	Agentic Engineering
Intent specification	Casual natural language prompts	Detailed prompts with examples and constraints	Formal specs, architecture docs, memory files
Verification	“Does it seem to work?”	Manual testing, spot-checking	Automated test suites, CI/CD gates, LM judges
Codebase understanding	Minimal; developer may not read the generated code	Selective review of critical paths	Comprehensive review of architecture; AI handles implementation details
Error handling	Copy-paste error messages back to the AI	Developer diagnoses root cause, AI implements fix	Agents self-diagnose within defined bounds; humans handle architectural issues
Appropriate scope	Prototypes, scripts, personal projects, hackathons	Features within established codebases	Production systems, team-scale development
Risk profile	High; acceptable for disposable code	Moderate; human judgment at key checkpoints	Low; systematic verification at every stage

Engineering is not just *programming*.

WHAT PROGRAMMING IS

One part of the job.

A bounded act. Hold the problem in your head, write the solution, make the tests pass. The scope is a session.

WHAT ENGINEERING IS

Keeping systems *working*.

Requirements, design, validation, change. Code is one ingredient. The system is the outcome. The scope is years.

WHAT
CHANGED

BEFORE

CODING

...

NOW

CODING

EVERYTHING ELSE

BREAKOUT • GROUPS OF 3-4

How do you use AI as an engineer?

- 01 Where in your work do you use it: planning, coding, debugging, tests, docs?
- 02 How do you check its output before you trust it?
- 03 One time it saved you, and one time it burned you.

[icon: Timer]

5

MINUTES

Then one takeaway
per group

What AI-native engineers actually do

BEFORE

Write first

Behavior, constraints, edge cases, acceptance criteria — before any prompt.

Set up context

Rules files and conventions the agent loads every time. Treat them as infrastructure.

DURING

Review against criteria

Intent, constraints, the existing system — not gut feel.

Test for agent mistakes

Edge cases, error paths, boundary conditions.

Iterate on purpose

Diagnose the gap before running it again.

THROUGHOUT

Own every commit

The agent is a collaborator, not a coauthor.

Stay close at high stakes

Migrations, architecture, production data.

Invest in context

Month 3 is easier than month 1.

Use AI for all the work

Docs, PRs, design exploration, explaining code.

THE SHIFT

Less time typing.
More time thinking, specifying,
reviewing and deciding.

BAD RESULT → REPHRASE AND HOPE

A random walk.

BAD RESULT → FIX THE GAP IN THE SPEC

A direct path to reliable output.

Skills that make you irreplaceable

Spec literacy

Write precise, agent-executable intent

Context engineering

Give the agent the right files, conventions and examples

Orchestration

Split work into scoped subtasks and integrate the results

Verification discipline

Review against criteria, not gut feel

Risk ownership

Calibrate oversight to what can go wrong

Systems thinking

Reason across service boundaries and over time

Communication & translation

Connect product intent and technical detail

Harness engineering

Build the environment that makes agent work reliable

5 emerging patterns

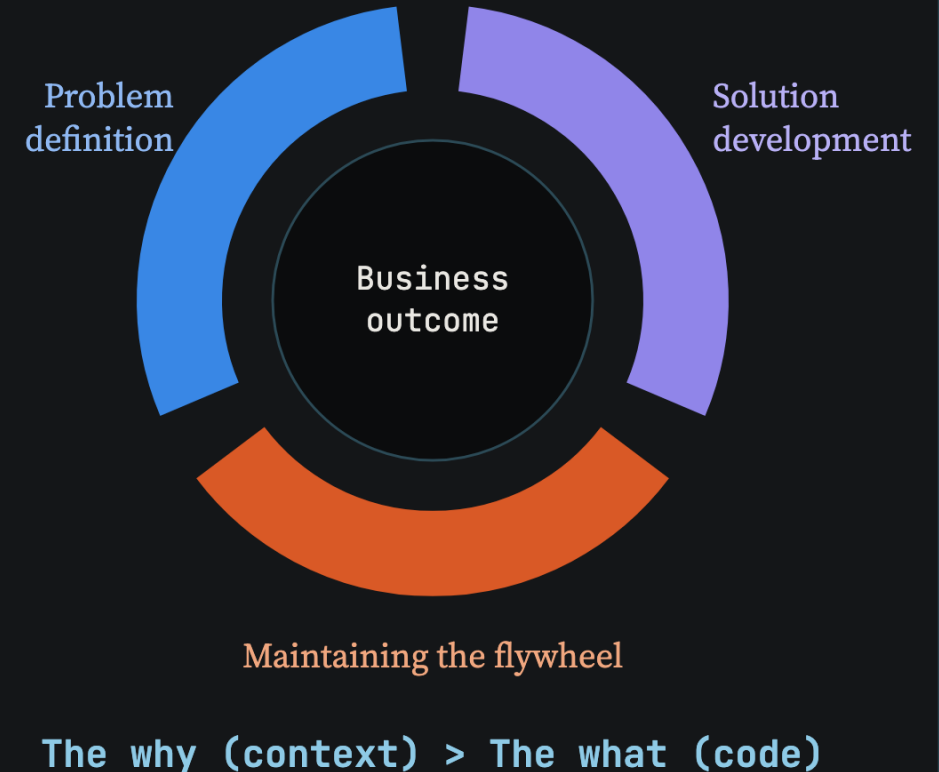
How AI-native engineers are thinking differently. Form Google -IO 2026

PATTERN 1 OF 5

Shift up: higher altitudes

*“Bridging into other areas of knowledge work.
Focusing more on the why vs the what.”*

Now that AI is accelerating execution, thinking deeply about **why** you’re building has never been more important.



PATTERN 2 OF 5

Shift left: structured intent

“Moving deep planning, context-setting earlier in the cycle. Creating new sources of truth in intent.”

A fundamental source of truth is moving away from code itself toward the **structured intent** that happens upfront.

```
# SYSTEM INTENT: User Dashboard Caching
**File:** `INTENT_SPEC.md`
**Author:** Lead Engineer & Product Manager
```

1. PRODUCT CONTEXT (The “Why”)

- **User Pain Point:** The analytics dashboard currently takes ~3.5 seconds to load, causing a 15% drop-off in user engagement during morning peak hours.
- **Business Goal:** Reduce P90 load times to under 500ms to rescue user retention.

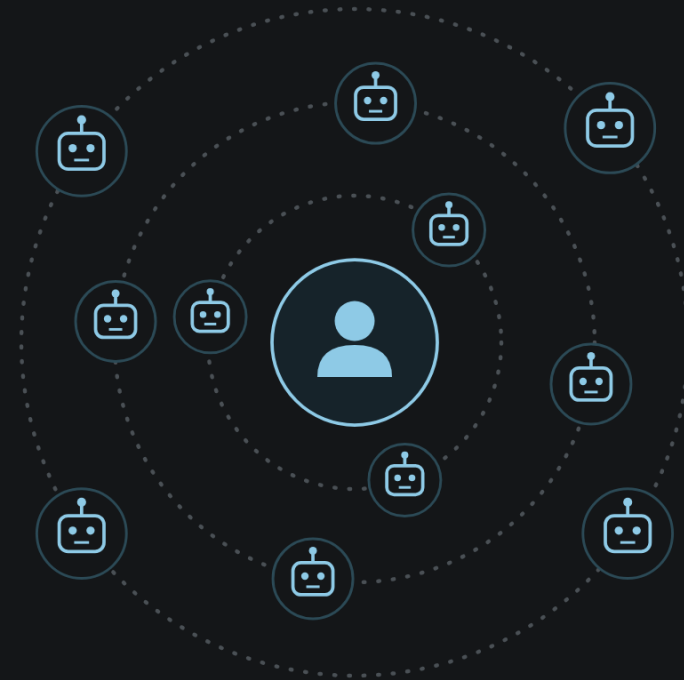
Intent > Implementation

PATTERN 3 OF 5

System design: the environment

“Creating environments that maximize the utility of AI. Designing for continuous feedback loops and validation.”

The best developers won't just be faster at coding — they'll be faster at **getting feedback and iterating** on solutions.



Verified output > Faster code

PATTERN 4 OF 5

System design: the workforce

“Evolving traditional job descriptions. Dissolving strict boundaries. Deploying agents autonomously.”

AI is decoupling a person's job role from the types of things they can do. **Small cross-functional teams execute at large scale.**



Capability > Job title

PATTERN 5 OF 5

System design: new values

*“Continually exploring at the cutting edge.
Prioritizing adaptability and celebrating learning.”*

Things move so fast — it’s about **learning what we can as we move**, then iterating and contributing to a learning organization.



Intelligent failure > The status quo

When AI writes the basics, how do juniors grow?

THE HONEST WORRY

AI now writes the boilerplate, glue code and standard tests — the work juniors used to learn from

Fewer junior roles, and a rising bar: “6 months’ experience + expert AI skills”

You can’t spot output that *looks* right but is wrong without hands-on practice

WHAT MATTERS MORE

The skills that turn a junior into a senior haven’t disappeared — they matter more

It’s easier than ever to be wrong in a way nobody notices

Be the engineer who catches the one wrong line in an otherwise correct PR

Habits that build a real foundation

01

Reach for the harder problem

Practice on problems a little beyond what's comfortable.

02

Go one layer deeper

Learn how your tools, frameworks and protocols actually work underneath.

03

Do the design work

Architecture and design — not only the implementation.

04

Read and write docs

Almost nobody enjoys it. Almost everybody learns from it.

05

Understand every line

Read the agent's code until you could have written it. Turn a shortcut into a tutor.

06

Own it, keep learning

Run with problems instead of waiting for every step. Never stop learning.

Sharpen the skills that sit above any technology

Problem-solving

Turn a vague, messy request into a problem you can actually attack.

Critical thinking

Reason from first principles instead of copying the nearest pattern. Ask “why” until the answer holds up.

Communication

Explain your thinking clearly — to teammates and to an agent — in plain language.

Working with AI

Manage context, shape the environment, steer the agent. Think in likelihoods: check the output, don’t trust one result.

None of this is new — and none of it is a nice-to-have.

The best learning accelerator juniors have ever had

AI AS YOUR TUTOR

Explore an unfamiliar codebase

Build real projects far faster

Ask endless “why does this work?” questions

THE NEW APPRENTICESHIP

Pair with a senior on the agent, not just the code

Learn when to trust it, rewrite it, or throw the prompt away

Choose your team well — you grow as fast as the people around you

The tools raised the floor. You still have to do the work — on purpose.

And this is what we'll learn
over the next 10 weeks.